University of BRISTOL

LECTURE 4

Neural networks

Foundations of Artificial Intelligence

SEMTM0051 · 2026/27

Lecturer: Dr Roberto Colomboni



Today’s lecture

Affine features and activation functions

Neural networks and the forward pass

What neural networks can represent

Fitting neural networks and local minima

Why neural networks?

So far, we have chosen the features
and fitted the output parameters.

$$f(x) = b + w_1 x + w_2 x^2$$

Fixed feature functions

$$\phi_1(x) = 1, \qquad \phi_2(x) = x, \qquad \phi_3(x) = x^2$$

Adjustable coefficients

$$b, \quad w_1, \quad w_2$$

Can we also learn the features from data?

Learning the feature functions

A neural network places adjustable parameters inside the feature functions.

$$f(\mathbf{x}) = b + \sum_{j=1}^{m} w_j \phi_j(\mathbf{x})$$

Feature functions with adjustable parameters

$\phi_1, \ldots, \phi_m$

Parameters combining the values of the feature functions

$b, w_1, \ldots, w_m$

How do we do that?

First, let us try affine feature functions with adjustable coefficients.

A first idea: affine feature functions

Make each feature map an affine function of the input.

$$\mathbf{x} = (x_1, \dots, x_d) \in \mathbb{R}^{1\times d}, \quad \mathbf{W} = \begin{pmatrix} W_{11} & \cdots & W_{1m} \\ \vdots & \ddots & \vdots \\ W_{d1} & \cdots & W_{dm} \end{pmatrix} \in \mathbb{R}^{d\times m}, \quad \mathbf{b} = (b_1, \dots, b_m) \in \mathbb{R}^{1\times m}$$

$$\phi_j(\mathbf{x}) = \underbrace{\sum_{k=1}^{d} x_k W_{kj}}_{\text{weighted sum}} + \underbrace{b_j}_{\text{bias}}, \qquad j = 1, \dots, m$$

$$\boldsymbol{\phi}(\mathbf{x}) = \big(\phi_1(\mathbf{x}), \dots, \phi_m(\mathbf{x})\big) = \mathbf{x}\mathbf{W} + \mathbf{b} \in \mathbb{R}^{1\times m}$$

The weights $\mathbf{W}$ and biases $\mathbf{b}$ determine the feature functions.

An affine feature and an affine output

The composition of two affine functions is an affine function.

Feature function

$$\phi(x) = w_1 x + b_1$$

$$w_1 = 2, \qquad b_1 = 1$$

Affine output

$$f(x) = w_2 \phi(x) + b_2$$

$$w_2 = 3, \qquad b_2 = -4$$

$$f(x) = 3(2x + 1) - 4 = 6x - 1$$

We can write the same predictor directly as an affine function of x, with no intermediate features:

$$f(x) = \tilde{w}x + \tilde{b}, \qquad \tilde{w} = 6, \quad \tilde{b} = -1$$

Affine features still give an affine predictor

Same problem for any number of affine features.

$$\boldsymbol{\phi}(\mathbf{x}) = \mathbf{x}\mathbf{W} + \mathbf{b}$$

Output weights and bias: $\mathbf{w}_{\text{out}} \in \mathbb{R}^{m\times 1}, \quad b_{\text{out}} \in \mathbb{R}$

$$f(\mathbf{x}) = \boldsymbol{\phi}(\mathbf{x})\mathbf{w}_{\text{out}} + b_{\text{out}}$$

$$f(\mathbf{x}) = \mathbf{x}\underbrace{\mathbf{W}\mathbf{w}_{\text{out}}}_{=:\tilde{\mathbf{w}}} + \underbrace{\mathbf{b}\mathbf{w}_{\text{out}} + b_{\text{out}}}_{=:\tilde{b}} = \mathbf{x}\tilde{\mathbf{w}} + \tilde{b}$$

f is still an affine function with equivalent weight column and bias:

$$\tilde{\mathbf{w}} := \mathbf{W}\mathbf{w}_{\text{out}} \in \mathbb{R}^{d\times 1}, \qquad \tilde{b} := \mathbf{b}\mathbf{w}_{\text{out}} + b_{\text{out}} \in \mathbb{R}$$

Repeating the argument: the composition of any number of affine maps remains affine.

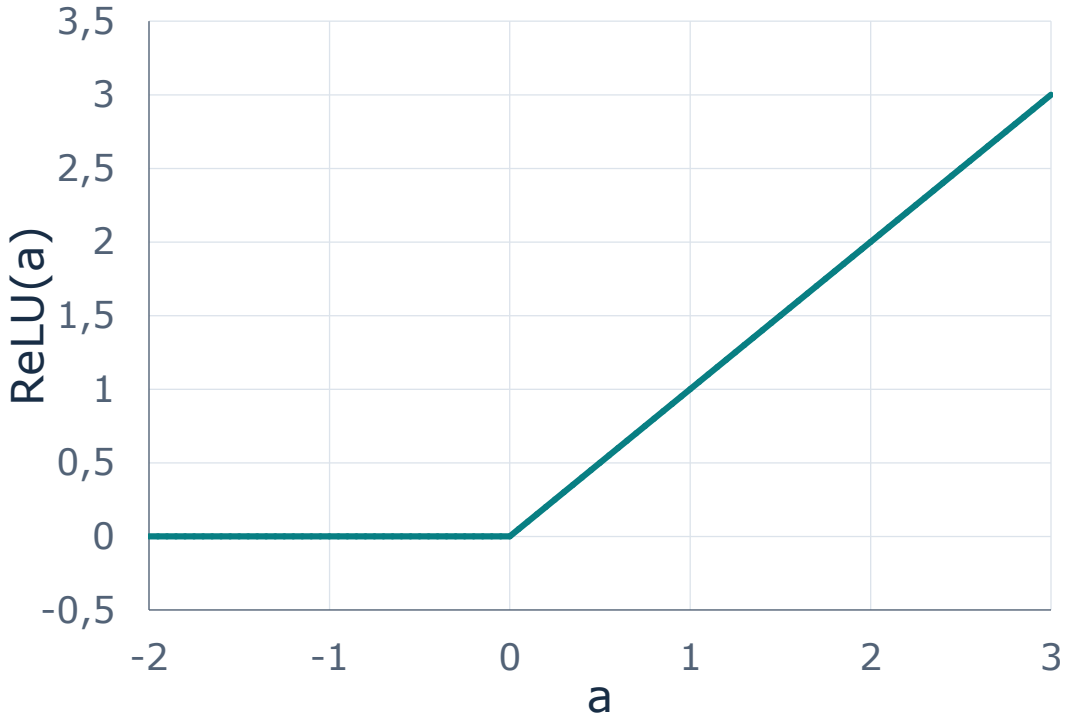
We need a nonlinear operation between the affine maps.

The ReLU activation function

We now apply a nonlinear function to each affine value.
We use ReLU (short for Rectified Linear Unit).

$$\mathrm{ReLU}(a) = \max\{0, a\}$$

$$= \begin{cases} 0 & a \leq 0 \\ a & a > 0 \end{cases}$$



Each feature combines an affine transformation with ReLU.

Today’s lecture

Affine features and activation functions

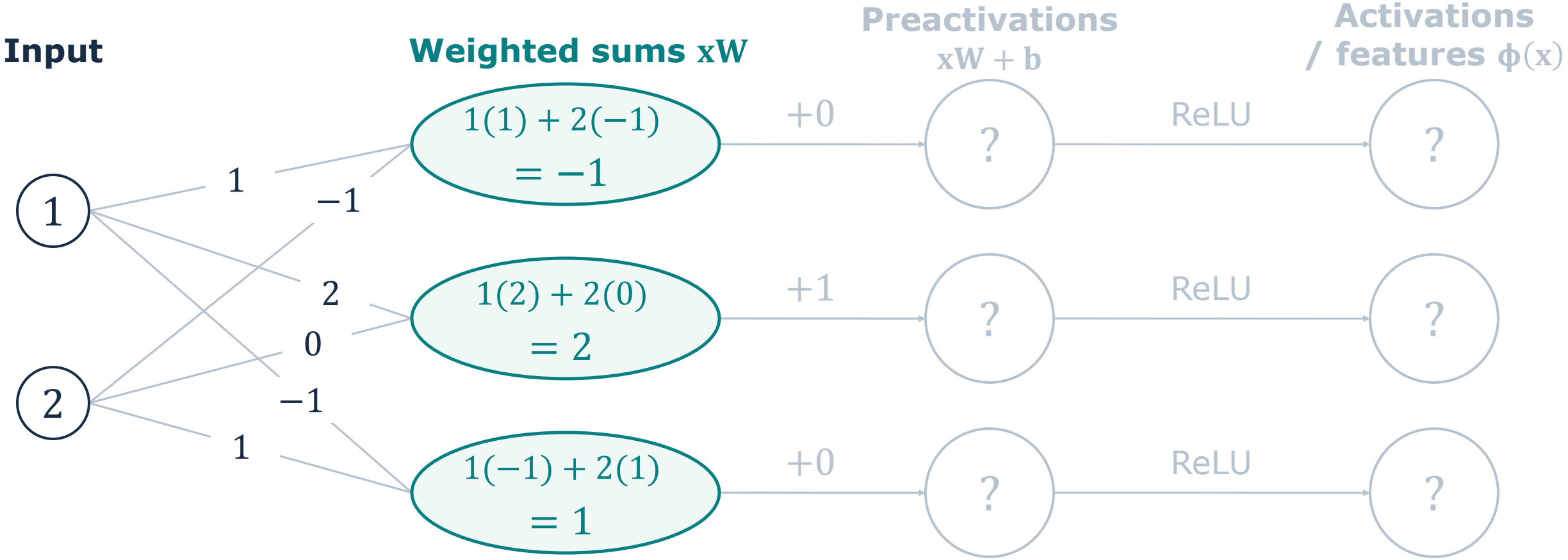
Neural networks and the forward pass

What neural networks can represent

Fitting neural networks and local minima

A first example – the weighted sum

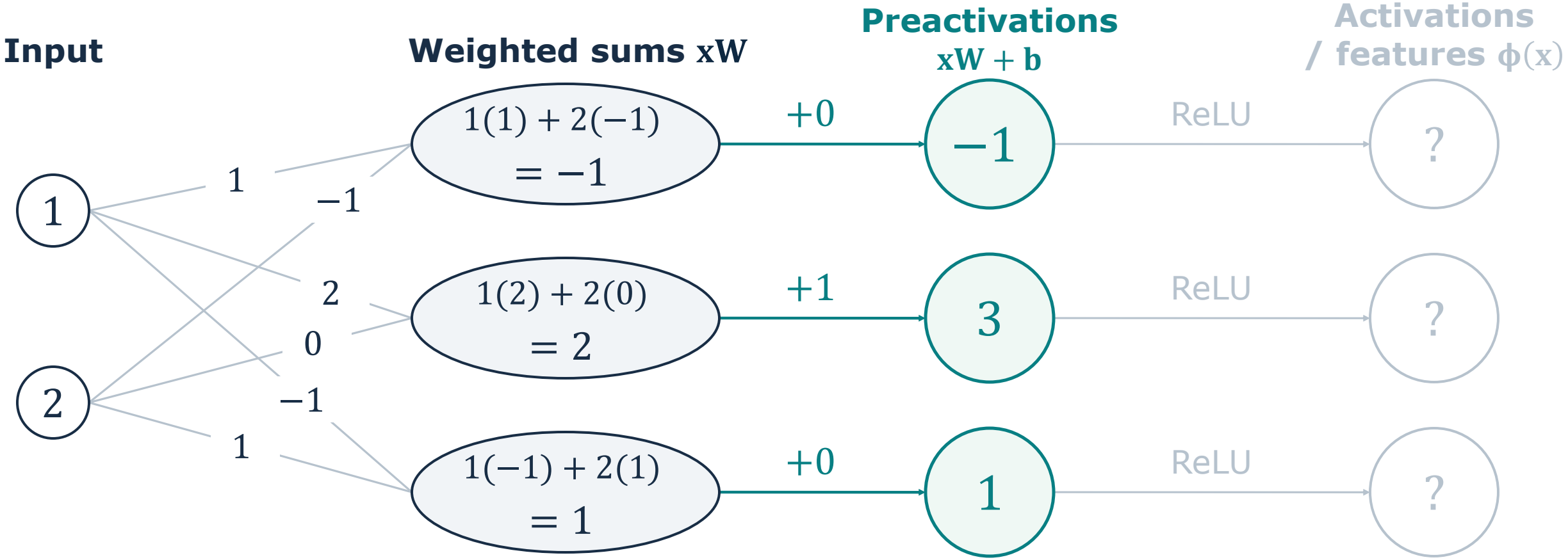
$$\mathbf{x} = (1,2), \quad \mathbf{W} = \begin{pmatrix} 1 & 2 & -1 \\ -1 & 0 & 1 \end{pmatrix}, \quad \mathbf{b} = (0,1,0)$$



$$\mathbf{xW} = (-1,2,1)$$

A first example – the preactivations

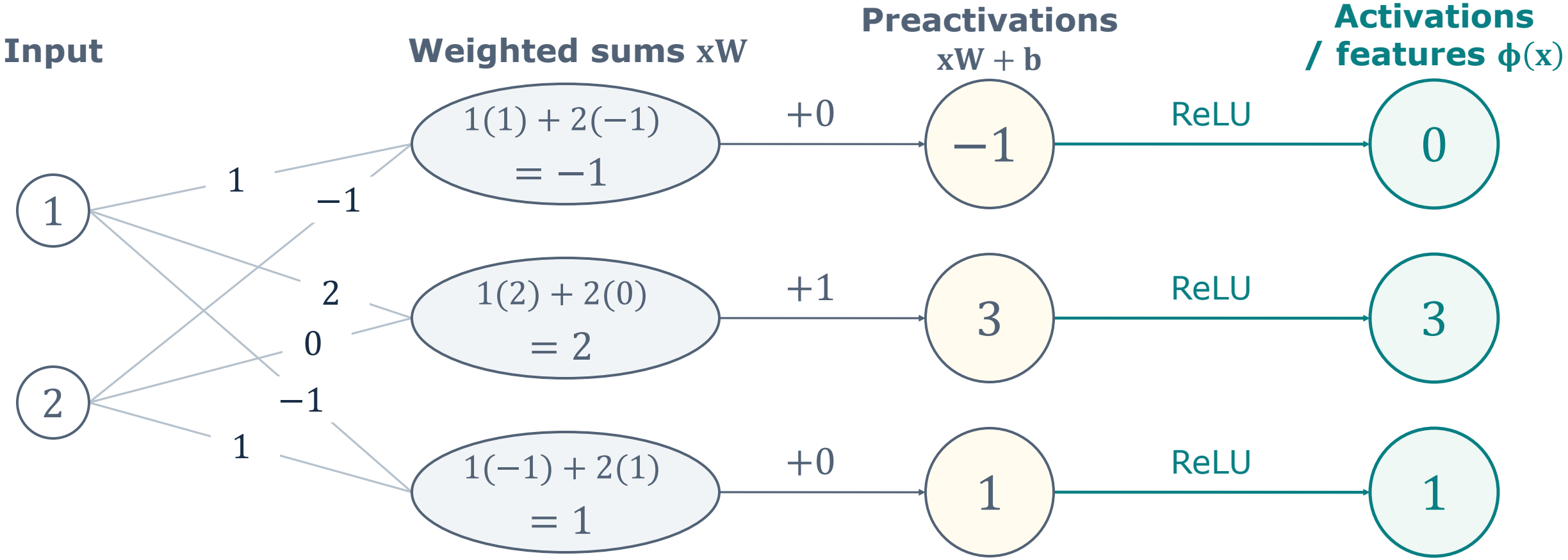
$$\mathbf{x} = (1,2), \quad \mathbf{W} = \begin{pmatrix} 1 & 2 & -1 \\ -1 & 0 & 1 \end{pmatrix}, \quad \mathbf{b} = (0,1,0)$$



$$\mathbf{xW} + \mathbf{b} = (-1,3,1)$$

A first example – the activations / features

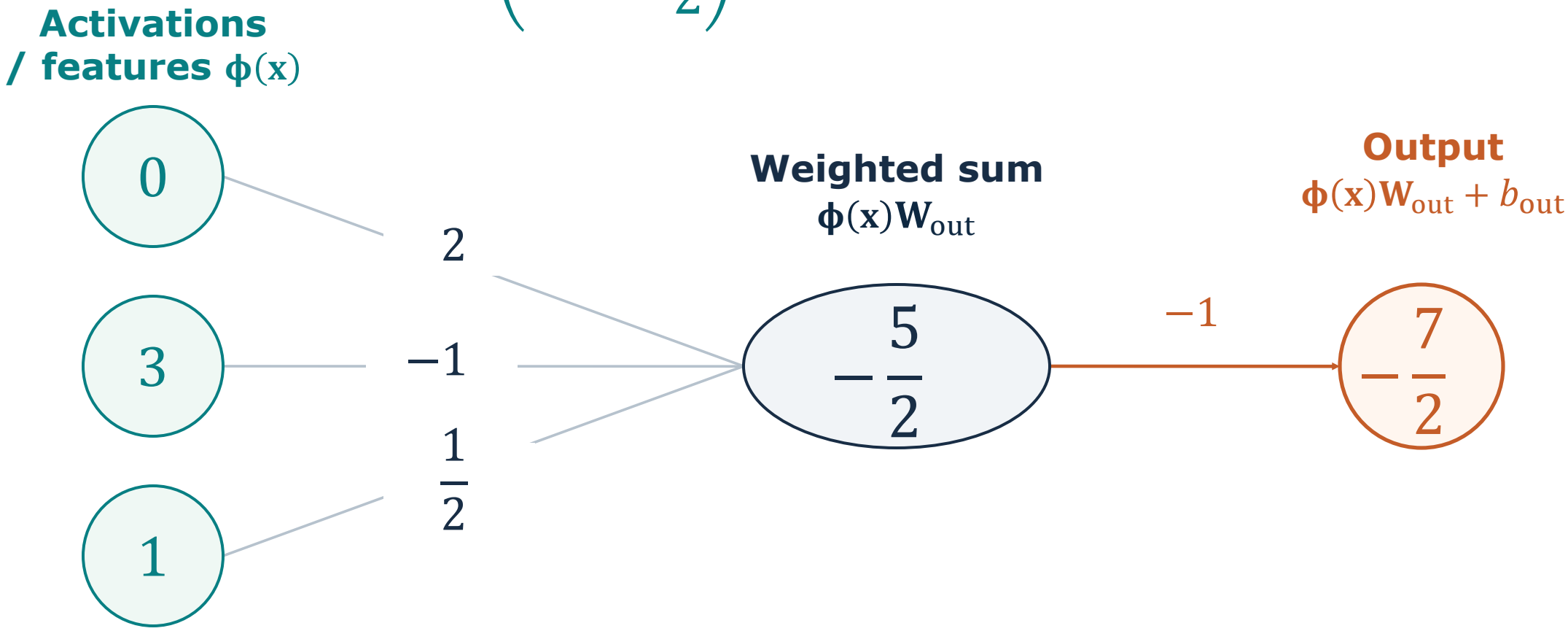
$$\mathbf{x} = (1,2), \quad \mathbf{W} = \begin{pmatrix} 1 & 2 & -1 \\ -1 & 0 & 1 \end{pmatrix}, \quad \mathbf{b} = (0,1,0)$$



$$\boldsymbol{\phi}(\mathbf{x}) = \left(\mathrm{ReLU}([\mathbf{xW}+\mathbf{b}]_1), \mathrm{ReLU}([\mathbf{xW}+\mathbf{b}]_2), \mathrm{ReLU}([\mathbf{xW}+\mathbf{b}]_3)\right) = (0,3,1)$$

A first example – the output

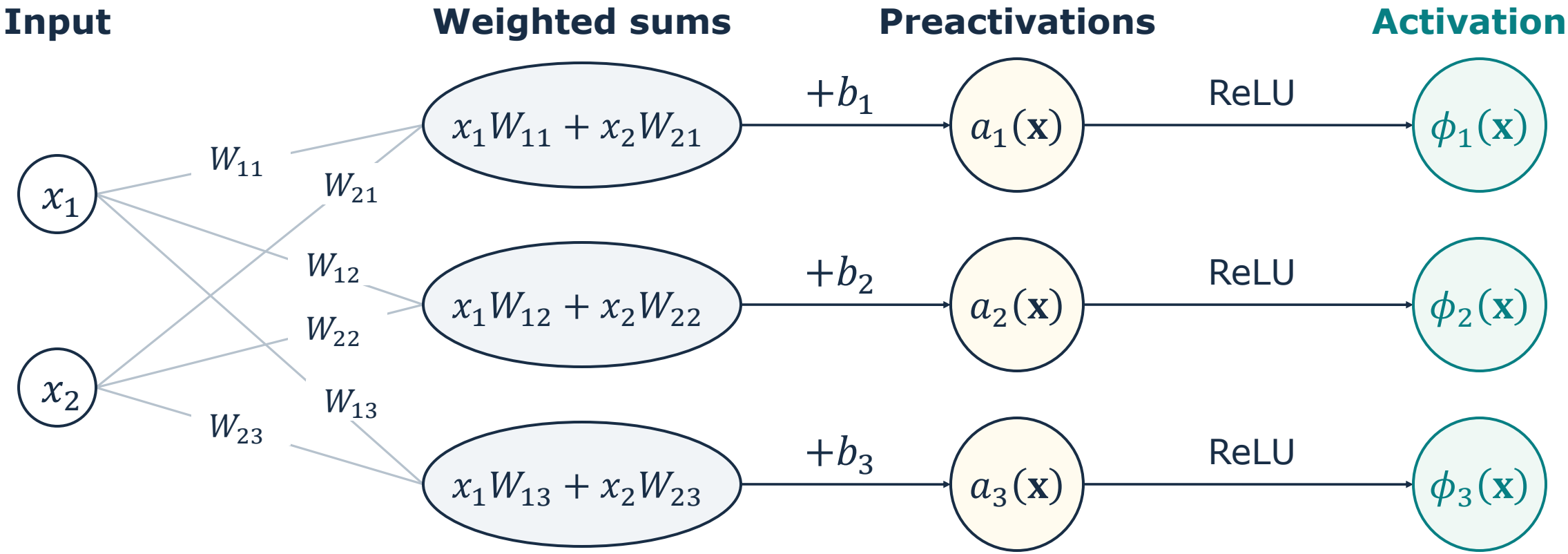
$$\mathbf{W}_{\text{out}} = \left(2, -1, \frac{1}{2}\right)^{\mathsf{T}}, \qquad b_{\text{out}} = -1$$



$$f\big((1,2)\big) = 0 \cdot (2) + 3 \cdot (-1) + 1 \cdot \left(\frac{1}{2}\right) - 1 = -\frac{7}{2}$$

Neurons

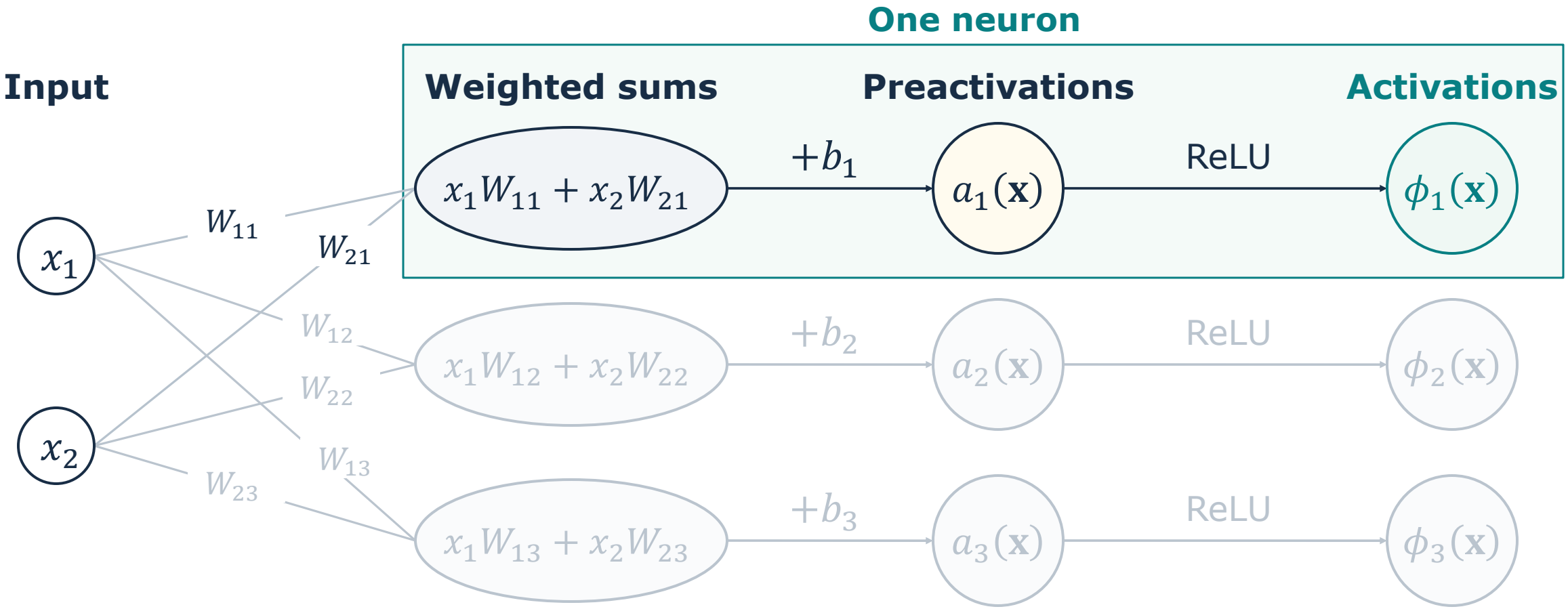
A neuron is a function that combines several inputs into one output.
The name is loosely inspired by biological neurons,
which receive signals from other cells and produce an output signal.



Each horizontal row represents one neuron.
It forms a weighted sum, adds a bias and applies an activation function.

A neuron

The highlighted row shows the complete calculation performed by one neuron.

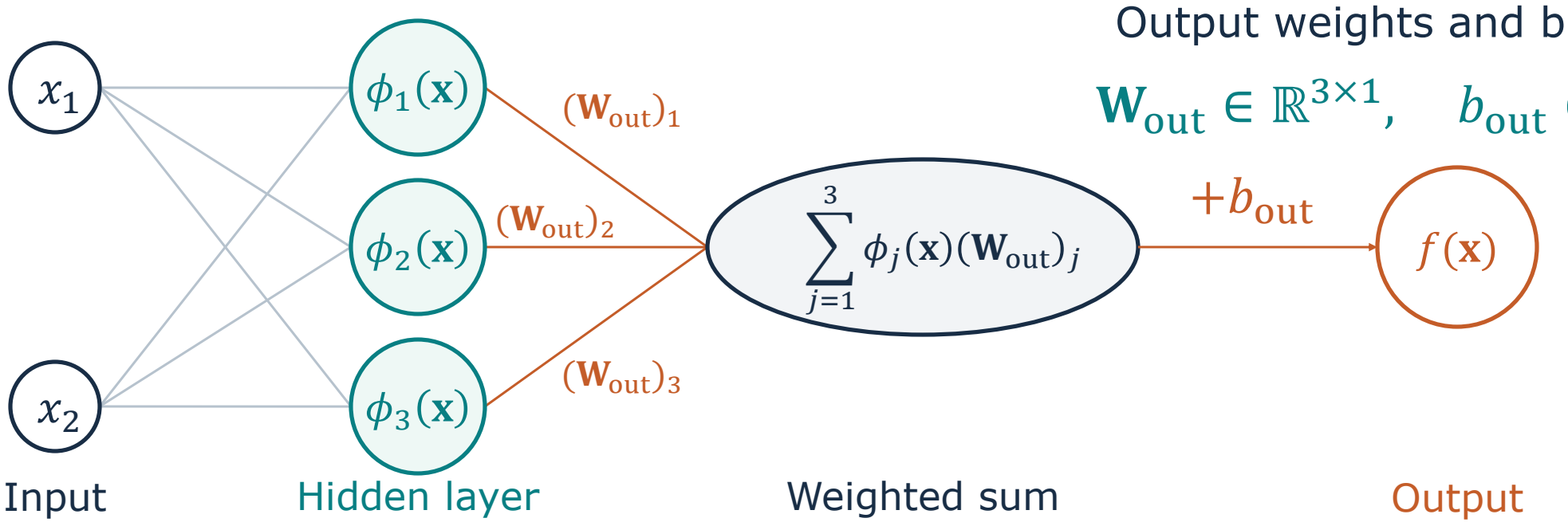


A neuron is a function that combines its inputs into one output, called its activation.

The output layer

Together, these neurons and the affine output layer form a neural network.
The output layer combines the neurons' activations using weights and a bias.

$$f(\mathbf{x}) = \sum_{j=1}^{3} \phi_j(\mathbf{x})(\mathbf{W}_{\text{out}})_j + b_{\text{out}} = \big(\phi_1(\mathbf{x}), \phi_2(\mathbf{x}), \phi_3(\mathbf{x})\big)\mathbf{W}_{\text{out}} + b_{\text{out}}$$



These neurons form a hidden layer:
their activations are intermediate values, computed inside the network.

A Neural Network with one hidden layer

A hidden layer groups neurons receiving the same input row.
Its width m is the number of neurons. Input and output dimensions are d and q.

$$\mathbf{x} = (x_1, \dots, x_d) \in \mathbb{R}^{1\times d}$$

$$\mathbf{W} \in \mathbb{R}^{d\times m}, \quad \mathbf{b} \in \mathbb{R}^{1\times m}, \quad \mathbf{W}_{\text{out}} \in \mathbb{R}^{m\times q}, \quad \mathbf{b}_{\text{out}} \in \mathbb{R}^{1\times q}$$

Preactivations: $\mathbf{a}(\mathbf{x}) \coloneqq \big(a_1(\mathbf{x}), \dots, a_m(\mathbf{x})\big) = \mathbf{x}\mathbf{W} + \mathbf{b}$

Activations: $\boldsymbol{\phi}(\mathbf{x}) \coloneqq \big(\phi_1(\mathbf{x}), \dots, \phi_m(\mathbf{x})\big) = \mathbf{ReLU}\big(\mathbf{a}(\mathbf{x})\big)$

where $\mathbf{ReLU}(\mathbf{a}) \coloneqq \big(\mathrm{ReLU}(a_1), \dots, \mathrm{ReLU}(a_m)\big)$

Output: $\mathbf{f}_{\boldsymbol{\theta}}(\mathbf{x}) \coloneqq \boldsymbol{\phi}(\mathbf{x})\mathbf{W}_{\text{out}} + \mathbf{b}_{\text{out}}$

The parameter column $\boldsymbol{\theta}$ collects all adjustable weights and biases.

Learning can change the feature functions

An illustrative change to the first neuron's weights and bias.

Original parameters

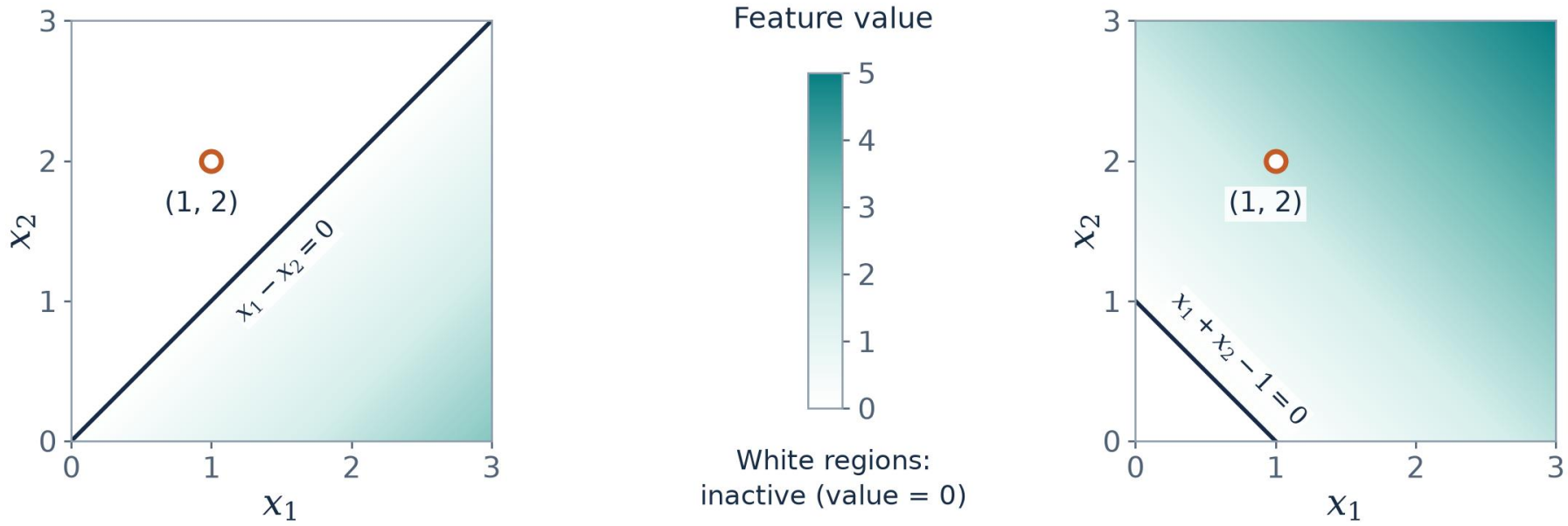
$$\phi_1(\mathbf{x}) = \mathrm{ReLU}(x_1 - x_2)$$

$$(W_{11}, W_{21}, b_1) = (1, -1, 0)$$

Changed parameters

$$\tilde{\phi}_1(\mathbf{x}) = \mathrm{ReLU}(x_1 + x_2 - 1)$$

$$(W_{11}, W_{21}, b_1) = (1, 1, -1)$$



$$\mathbf{x} = (1,2): \quad \phi_1(\mathbf{x}) = 0 \quad \longrightarrow \quad \tilde{\phi}_1(\mathbf{x}) = 2$$

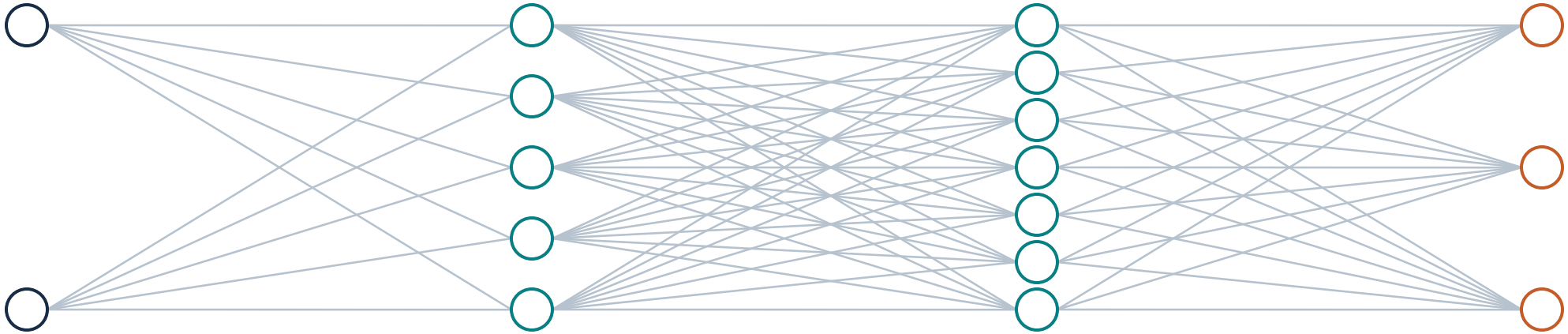
The input stays the same, but the feature value changes.

Neural Network with multiple hidden layers

Width m_r: the number of neurons in layer r.

By convention, $m_0 = d$ is the input dimension and $m_L = q$ the output dimension.

Depth L: the number of layers, counting the output but excluding the input.



Input:
$m_0 = d = 2$ coordinates

Hidden layer 1:
width $m_1 = 5$

Hidden layer 2:
width $m_2 = 7$

Output layer $L = 3$:
width $m_3 = q = 3$

Each additional hidden layer uses the previous layer's activations as inputs.

The input dimension and layer widths specify the architecture: **2 → 5 → 7 → 3.**

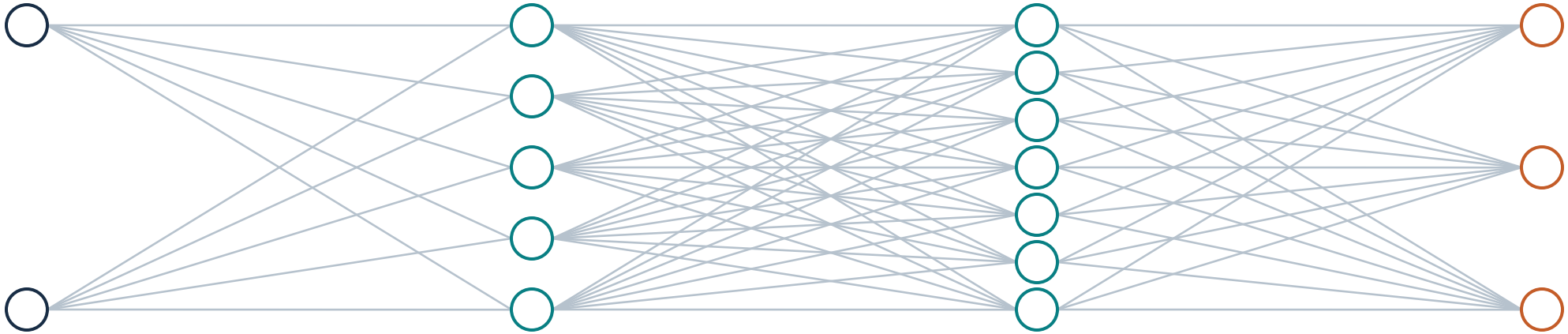
The architecture fixes parameter shapes; fitting adjusts their values.

Neural Network with multiple hidden layers

Width m_r: the number of neurons in layer r.

By convention, $m_0 = d$ is the input dimension and $m_L = q$ the output dimension.

Depth L: the number of layers, counting the output but excluding the input.



Input:
$m_0 = d = 2$ coordinates

Hidden layer 1:
width $m_1 = 5$

Hidden layer 2:
width $m_2 = 7$

Output layer $L = 3$:
width $m_3 = q = 3$

Input: $\boldsymbol{\phi}_0(\mathbf{x}) = \mathbf{x},$

Layer r preactivations: $\mathbf{a}_r(\mathbf{x}) \coloneqq \boldsymbol{\phi}_{r-1}(\mathbf{x})\mathbf{W}_r + \mathbf{b}_r \qquad r = 1, \ldots, L-1$

Layer r activations : $\boldsymbol{\phi}_r(\mathbf{x}) \coloneqq \mathbf{ReLU}\big(\mathbf{a}_r(\mathbf{x})\big) \qquad r = 1, \ldots, L-1$

Output: $\mathbf{f}_{\boldsymbol{\theta}}(\mathbf{x}) \coloneqq \boldsymbol{\phi}_{L-1}(\mathbf{x})\mathbf{W}_L + \mathbf{b}_L$

Dimensions and parameters

For the architecture $2 \to 5 \to 7 \to 3$,
each layer r has its own weights and biases.

$$\mathbf{W}_r \in \mathbb{R}^{m_{r-1} \times m_r}, \qquad \mathbf{b}_r \in \mathbb{R}^{1 \times m_r}, \qquad r = 1, \ldots, L$$

Output parameters: $\mathbf{W}_L = \mathbf{W}_{\text{out}}$ and $\mathbf{b}_L = \mathbf{b}_{\text{out}}$.

Layer	Weight matrix	Bias row	Parameters
1	2 × 5	1 × 5	10 + 5 = 15
2	5 × 7	1 × 7	35 + 7 = 42
3	7 × 3	1 × 3	21 + 3 = 24

$$15 + 42 + 24 = 81$$

The parameter column θ has 81 entries.

Regression and binary classification

The same scalar network output can play different roles.

Regression

The network output $f_{\boldsymbol{\theta}}(\mathbf{x})$ is the predicted value.

$$\hat{y} = f_{\boldsymbol{\theta}}(\mathbf{x})$$

$$\ell_{\text{sq}}(\hat{y}, y) = (\hat{y} - y)^2$$

Use the squared loss.

Binary classification

The network output $f_{\boldsymbol{\theta}}(\mathbf{x})$ is a score.

$$p_{\boldsymbol{\theta}}(\mathbf{x}) = \sigma\big(f_{\boldsymbol{\theta}}(\mathbf{x})\big)$$

$$h_{p_{\boldsymbol{\theta}}}(\mathbf{x}) = \begin{cases} 1 & p_{\boldsymbol{\theta}}(\mathbf{x}) > 1/2 \\ 0 & p_{\boldsymbol{\theta}}(\mathbf{x}) \leq 1/2 \end{cases}$$

$$\ell_{\log}(p, y) = y \log\left(\frac{1}{p}\right) + (1 - y) \log\left(\frac{1}{1 - p}\right)$$

Use the logistic loss.

For each task, neural networks use the same loss as the corresponding linear model.

Multiclass classification

With K classes, the network outputs a row of K real scores.
The output width is $m_L = q = K$.

$$\mathbf{f_\theta}(\mathbf{x}) \in \mathbb{R}^{1\times K}$$

$$\mathbf{p_\theta}(\mathbf{x}) = \mathrm{Softmax}\big(\mathbf{f_\theta}(\mathbf{x})\big) \in (0,1)^{1\times K}$$

Use the plug-in classifier $h_{\mathbf{p_\theta}}(\mathbf{x}) \in \{1, \dots, K\}$ associated with $\mathbf{p_\theta}(\mathbf{x})$:
choose the most probable class, breaking ties with the smallest index.

Fit with the same cross-entropy loss

$$\ell_{\mathrm{multi}}(\mathbf{p}, y) = -\log\big(p_y\big)$$

Today’s lecture

Affine features and activation functions

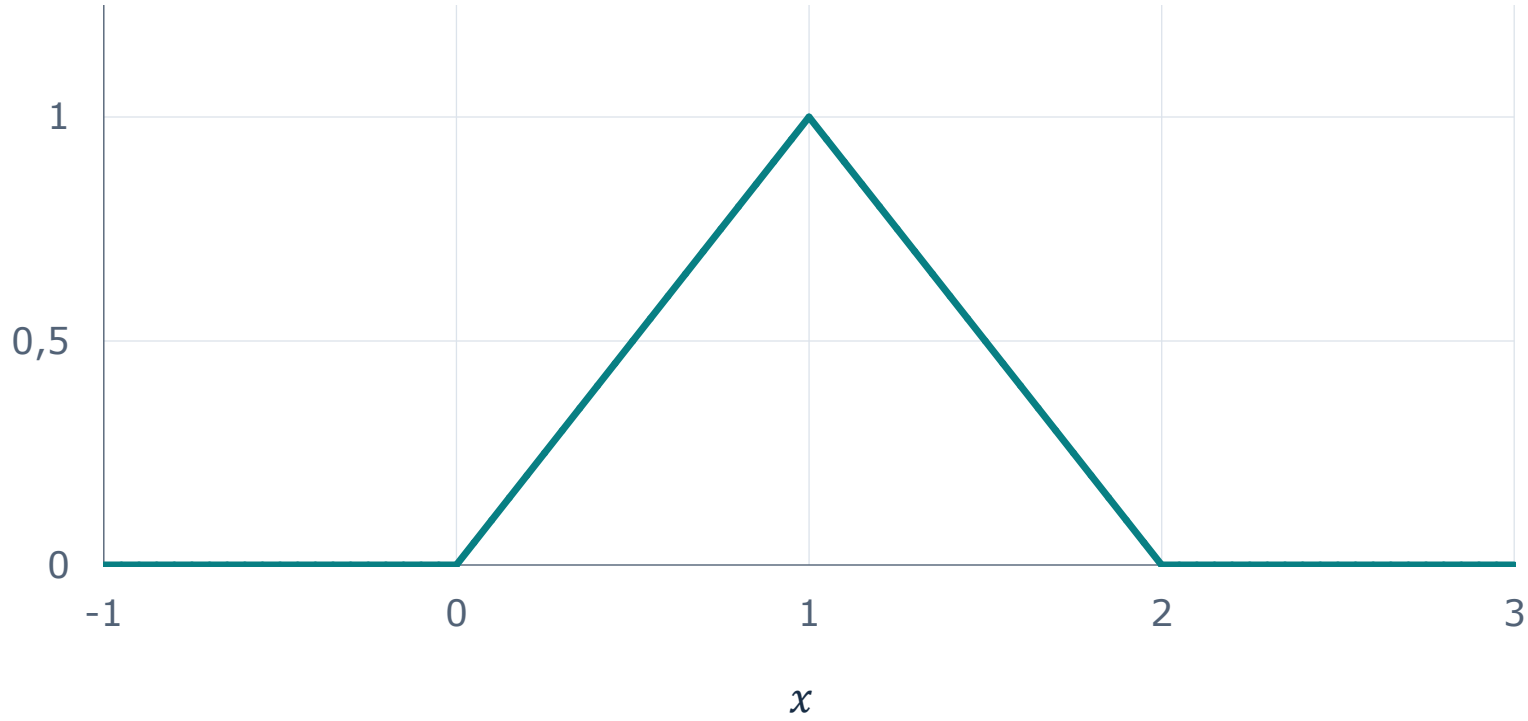
Neural networks and the forward pass

What neural networks can represent

Fitting neural networks and local minima

A regression problem: the tent function

We could choose this function as a fixed feature.

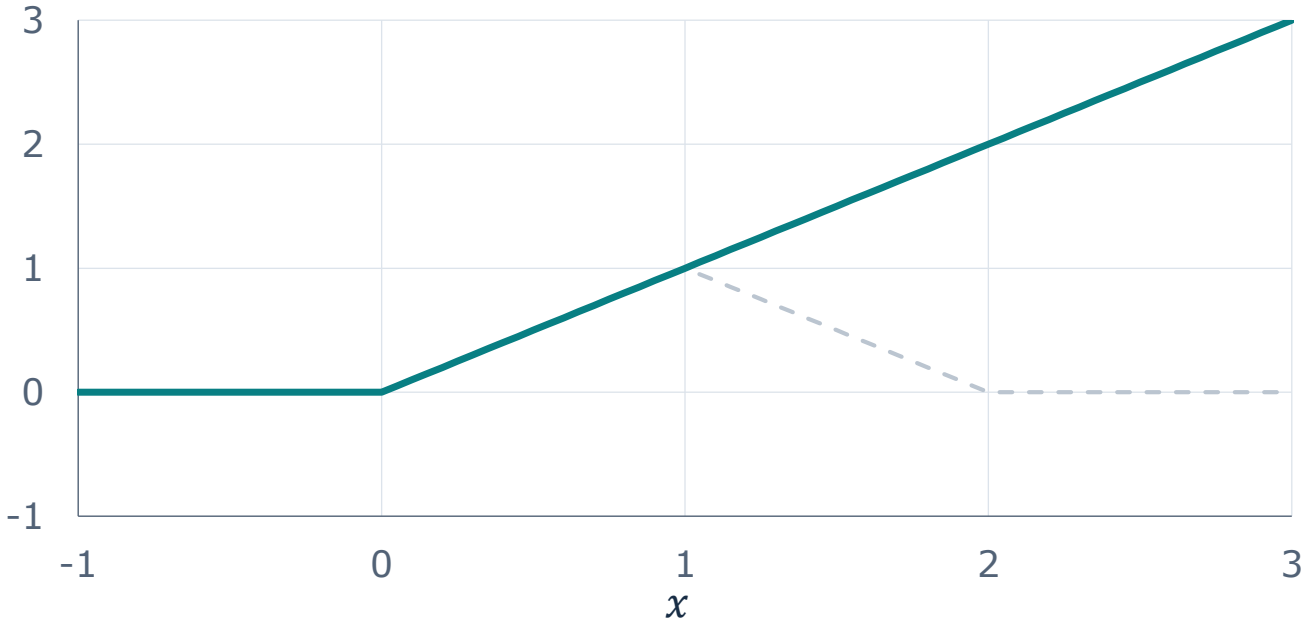


Can we build it
by adjusting the parameters of a Neural Network?

Building a tent function: start

At $x = 0$, the slope changes from 0 to $+1$.

$$T_1(x) = \mathrm{ReLU}(x)$$



One neuron
width $m_1 = 1$

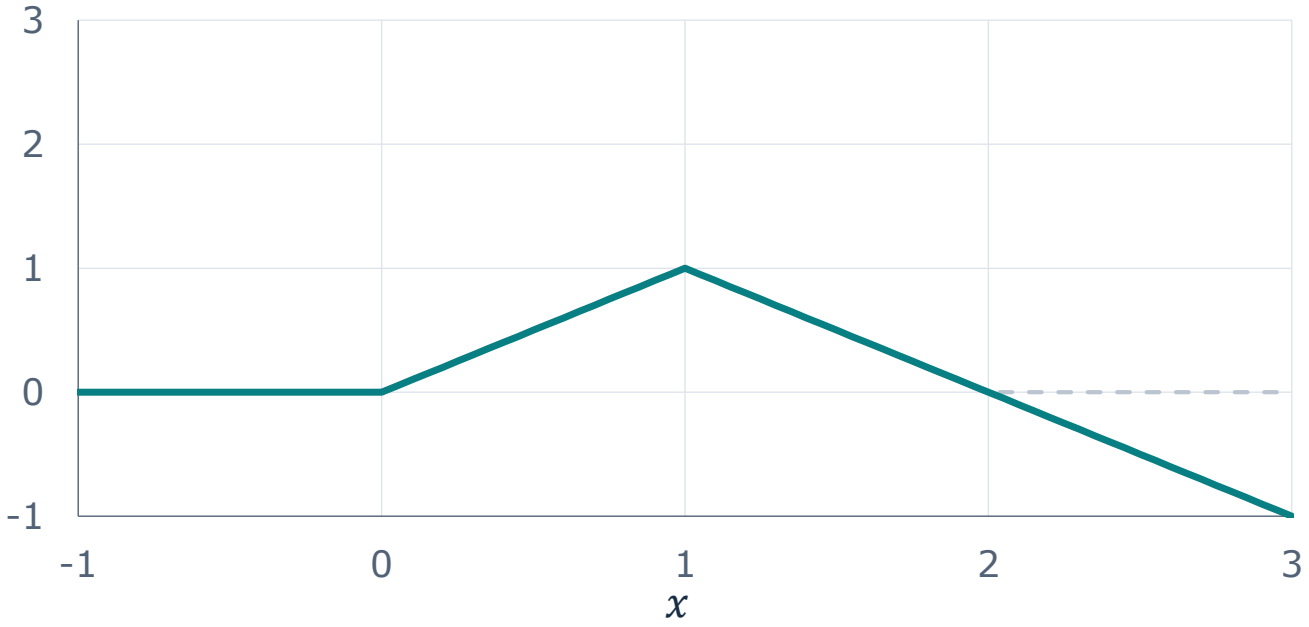
The target returns to zero.

One ReLU creates one change of slope.

Building a tent function: turn

At $x = 1$, the second neuron changes the slope by -2.

$$T_2(x) = \mathrm{ReLU}(x) - 2\mathrm{ReLU}(x-1)$$



Two neurons
width $m_1 = 2$

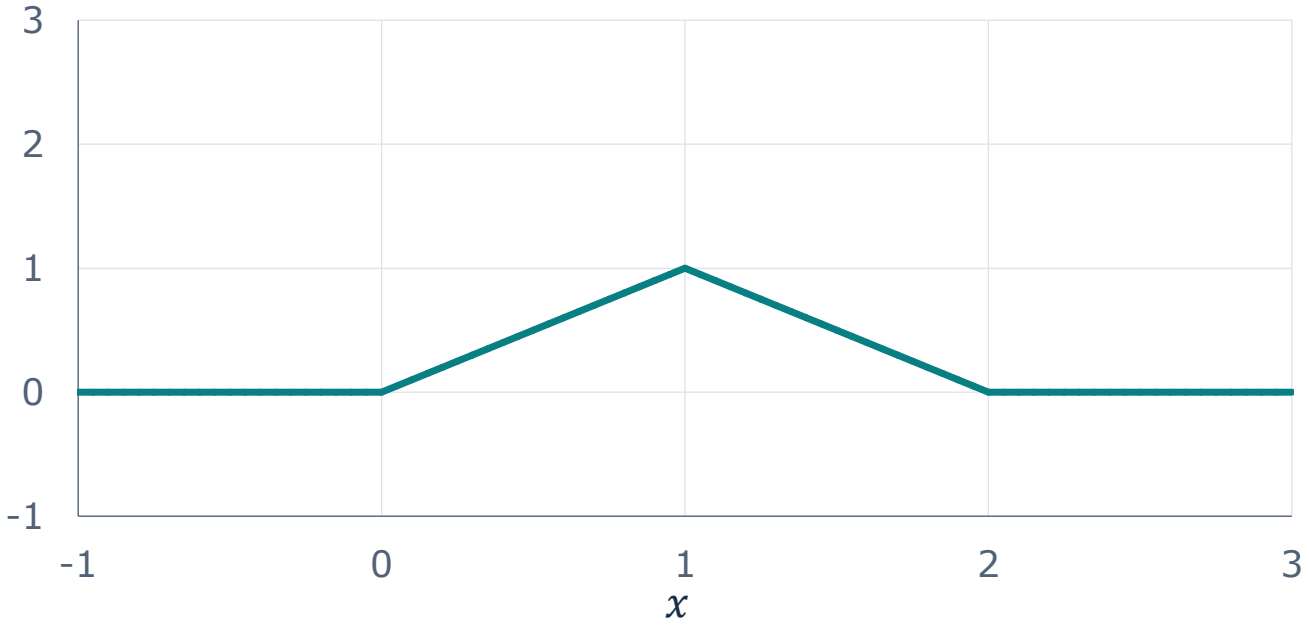
The slope is now -1,
but must become zero.

The output weights can be negative.

Building a tent function: stop

At $x = 2$, the third neuron changes the slope by $+1$.

$$T(x) = \mathrm{ReLU}(x) - 2\mathrm{ReLU}(x-1) + \mathrm{ReLU}(x-2)$$



Three neurons
width $m_1 = 3$

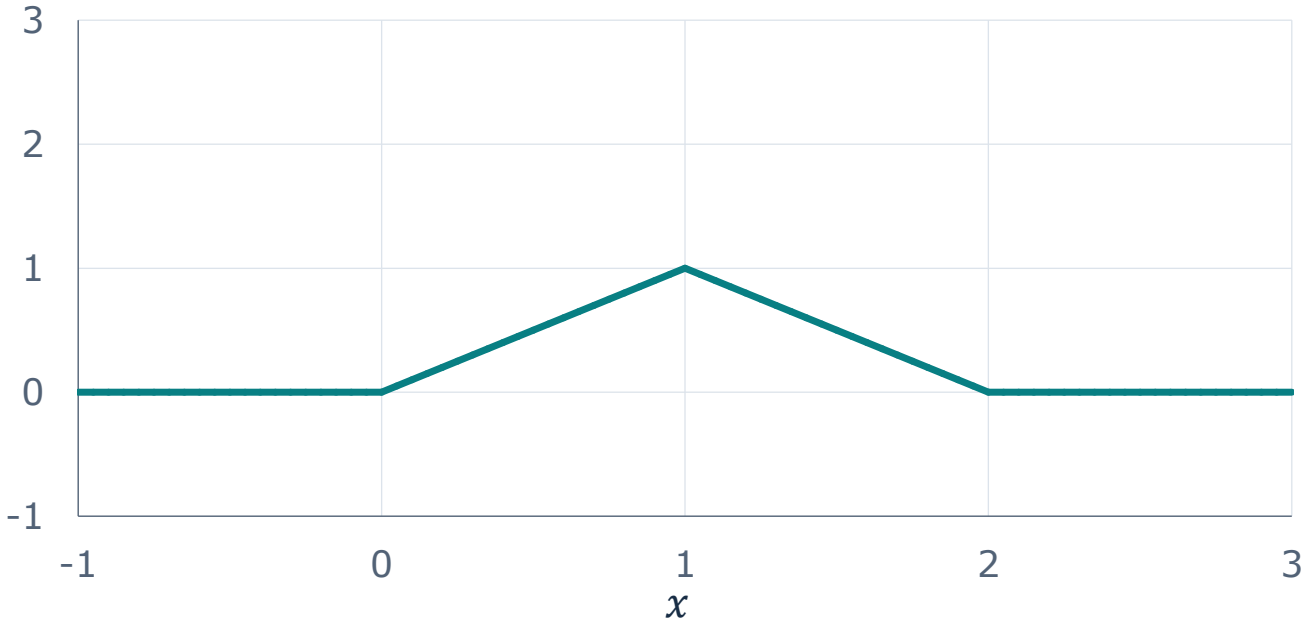
The slope returns to zero.
The tent is complete.

Three neurons can produce the tent function.

The parameters of the tent network

The tent function can be represented by a neural network with architecture: $1 \to 3 \to 1$

$$T(x) = 1 \cdot \mathrm{ReLU}(1 \cdot x + 0) - 2 \cdot \mathrm{ReLU}(1 \cdot x + (-1)) + 1 \cdot \mathrm{ReLU}(1 \cdot x + (-2)) + 0$$



$$\mathbf{W}_1 = (1,1,1)$$

$$\mathbf{b}_1 = (0,-1,-2)$$

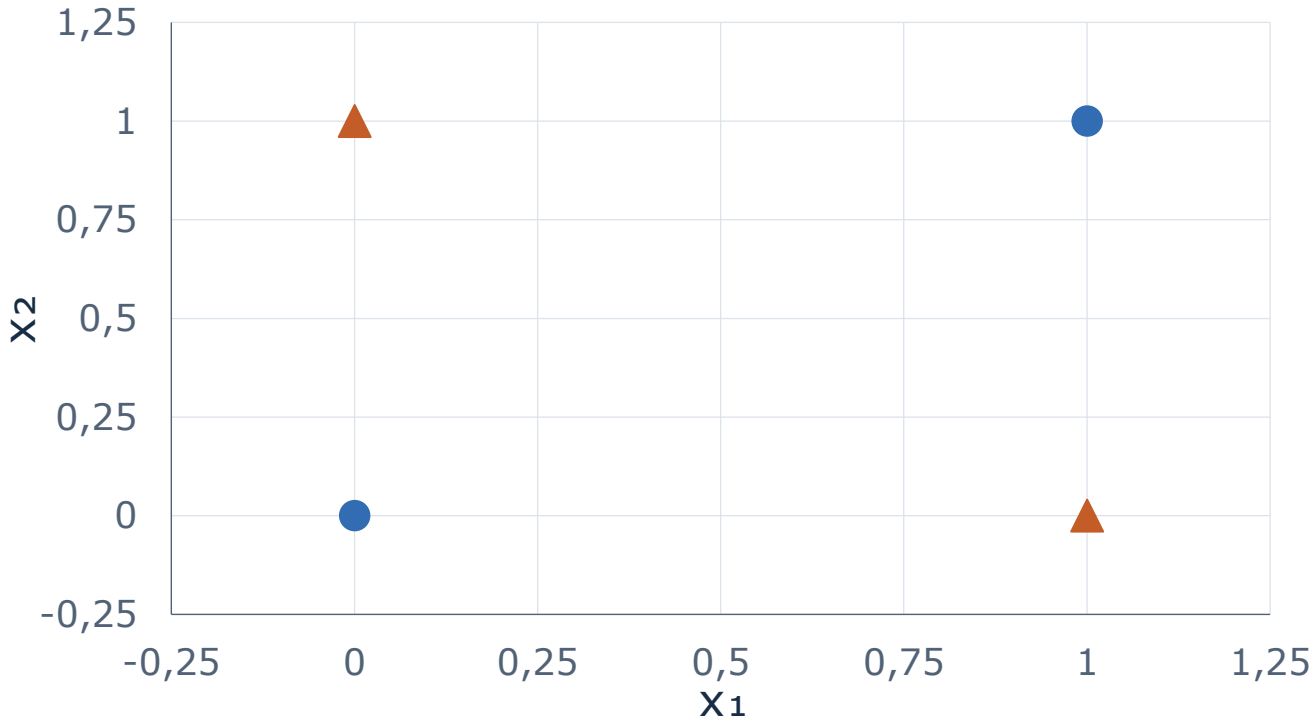
$$\mathbf{W}_2 = (1,-2,1)^\mathsf{T}$$

$$\mathbf{b}_2 = (0)$$

The biases place the breakpoints. The output weights change the slopes.

A classification problem: the XOR function

The label is 1 when exactly one input coordinate is 1.



$$(0,0) \mapsto 0, \quad (1,1) \mapsto 0$$

$$(1,0) \mapsto 1, \quad (0,1) \mapsto 1$$

Can one straight-line decision boundary classify all four observations?

Why an affine score cannot solve XOR

The score must be positive for class 1 and nonpositive for class 0.

$$s(\mathbf{x}) = b + w_1 x_1 + w_2 x_2$$

Class 0 observations

$$b = s(0,0) \leq 0$$
$$b + w_1 + w_2 = s(1,1) \leq 0$$

Class 1 observations

$$b + w_1 = s(1,0) > 0$$
$$b + w_2 = s(0,1) > 0$$

Adding the inequalities within each class gives two incompatible inequalities.

$$2b + w_1 + w_2 \leq 0$$

$$2b + w_1 + w_2 > 0$$

Two features that solve XOR

The XOR function can be represented by a neural network with architecture: $2 \to 2 \to 1$

$$\phi_1(\mathbf{x}) = \mathrm{ReLU}(x_1 - x_2), \qquad \phi_2(\mathbf{x}) = \mathrm{ReLU}(x_2 - x_1)$$

$$f_{\boldsymbol{\theta}}(\mathbf{x}) = \phi_1(\mathbf{x}) + \phi_2(\mathbf{x}) - \frac{1}{2} = |x_1 - x_2| - \frac{1}{2}$$

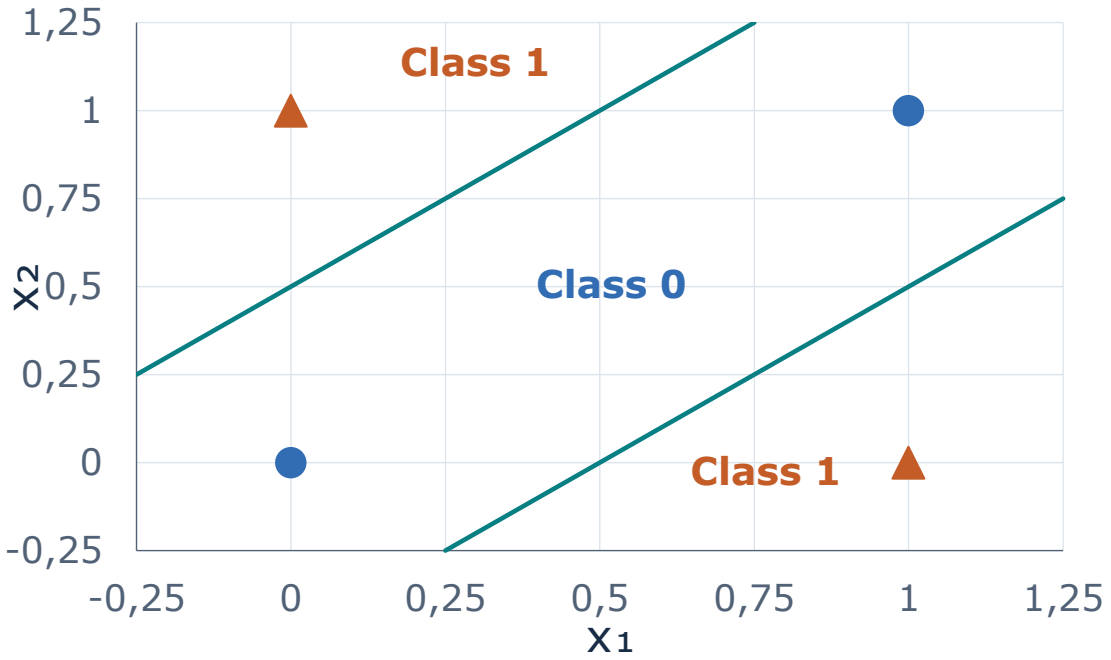
$$\mathbf{W}_1 = \begin{pmatrix} 1 & -1 \\ -1 & 1 \end{pmatrix}, \quad \mathbf{b}_1 = (0,0), \qquad \mathbf{W}_2 = \begin{pmatrix} 1 \\ 1 \end{pmatrix}, \quad \mathbf{b}_2 = \begin{pmatrix} -\frac{1}{2} \end{pmatrix}$$

Input row	Feature row	Score	Label
(0, 0)	(0, 0)	−0.5	0
(1, 0)	(1, 0)	+0.5	1
(0, 1)	(0, 1)	+0.5	1
(1, 1)	(0, 0)	−0.5	0

A straight boundary in feature space

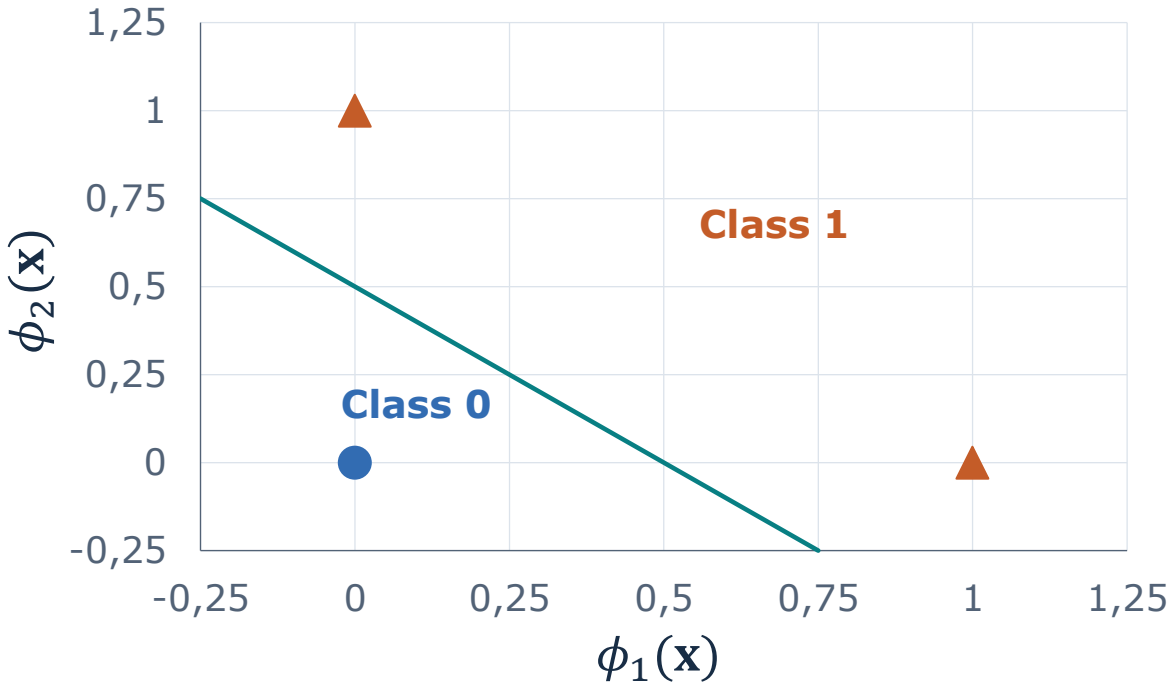
Non-affine features allow an affine output to produce
a nonlinear decision boundary in the original input space.

Input space



$$|x_1 - x_2| = \frac{1}{2}$$

Feature space



$$\phi_1(\mathbf{x}) + \phi_2(\mathbf{x}) = \frac{1}{2}$$

$(0,0)$ and $(1,1)$ both map to the origin.

Today’s lecture

Affine features and activation functions

Neural networks and the forward pass

What neural networks can represent

Fitting neural networks and local minima

Fitting all weights and biases

The observations stay fixed. The network parameters vary.

Regression

$$\mathcal{L}_{\text{sq}}(\boldsymbol{\theta}) = \sum_{i=1}^{N} \ell_{\text{sq}}(f_{\boldsymbol{\theta}}(\mathbf{x}_i), y_i)\,,$$

Binary Classification

$$\mathcal{L}_{\log}(\boldsymbol{\theta}) = \sum_{i=1}^{N} \ell_{\log}(p_{\boldsymbol{\theta}}(\mathbf{x}_i), y_i)$$

Let $\mathcal{L}$ be the empirical loss for the task.
Where differentiable, we update the parameters by gradient descent.

$$\boldsymbol{\theta}^{(t+1)} = \boldsymbol{\theta}^{(t)} - \eta \nabla \mathcal{L}\left(\boldsymbol{\theta}^{(t)}\right), \qquad \eta > 0$$

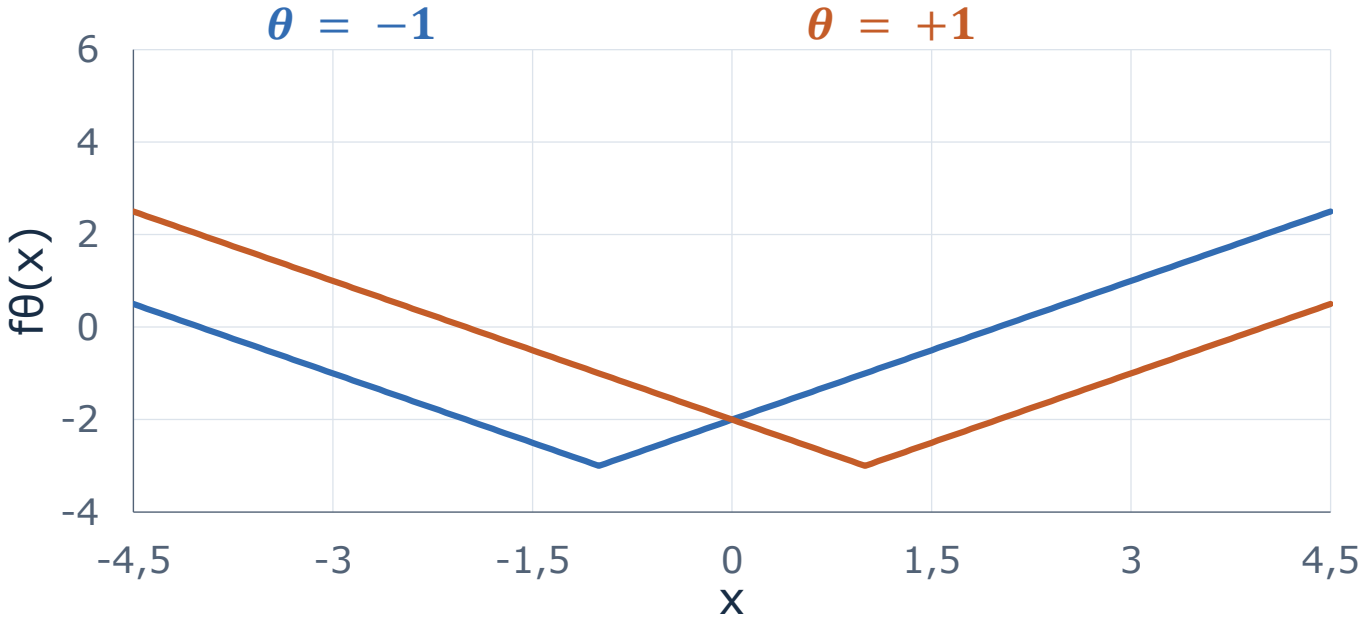
For neural networks,
will gradient descent converge to a global minimum?

A network with one adjustable parameter

To visualise the empirical loss, we vary only one scalar parameter θ.
The hidden biases are $-\theta$ and θ. All other coefficients stay fixed.

$L = 2$, with widths $m_0 = 1$, $m_1 = 2$ and $m_2 = 1$.

$$f_\theta(x) = \mathrm{ReLU}(x - \theta) + \mathrm{ReLU}(\theta - x) - 3 = |x - \theta| - 3$$



Hidden bias row:

$$\mathbf{b}_1 = (-\theta, \theta)$$

The two biases vary together and remain opposites.

Changing θ shifts the graph horizontally.

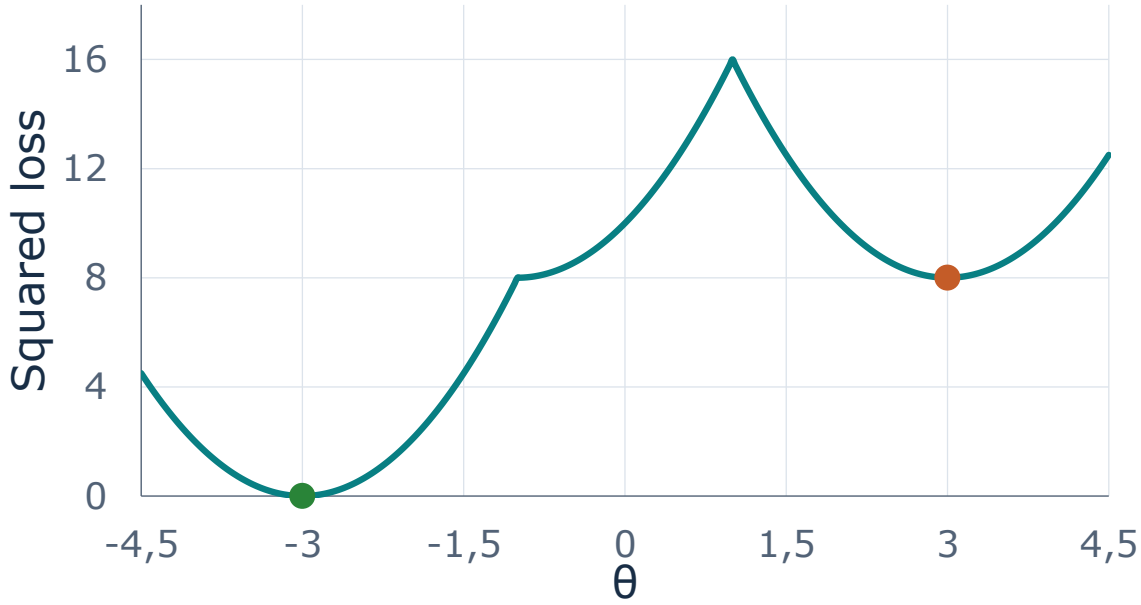
Similar optimisation difficulties can also occur in networks where all parameters can vary.

The regression loss

Two observations: $(x_1, y_1) = (-1, -1)$ and $(x_2, y_2) = (1, 1)$.

$$\mathcal{L}_{\text{sq}}(\theta) = (f_\theta(-1) + 1)^2 + (f_\theta(1) - 1)^2$$

$$\mathcal{L}_{\text{sq}}(\theta) = \begin{cases} 2(\theta + 3)^2 & \theta \leq -1 \\ 2(\theta + 1)^2 + 8 & -1 < \theta < 1 \\ 2(\theta - 3)^2 + 8 & \theta \geq 1 \end{cases}$$



The loss is a function of θ. Its graph has two valleys.

Two stationary points, different losses

Both points have derivative zero, but their losses are different.

Parameter	Predictions at $x = -1$ and 1	Squared loss
$\theta = -3$	-1 and 1	0
$\theta = +3$	1 and -1	8

Global minimum at $\theta = -3$

No parameter value has a smaller loss.

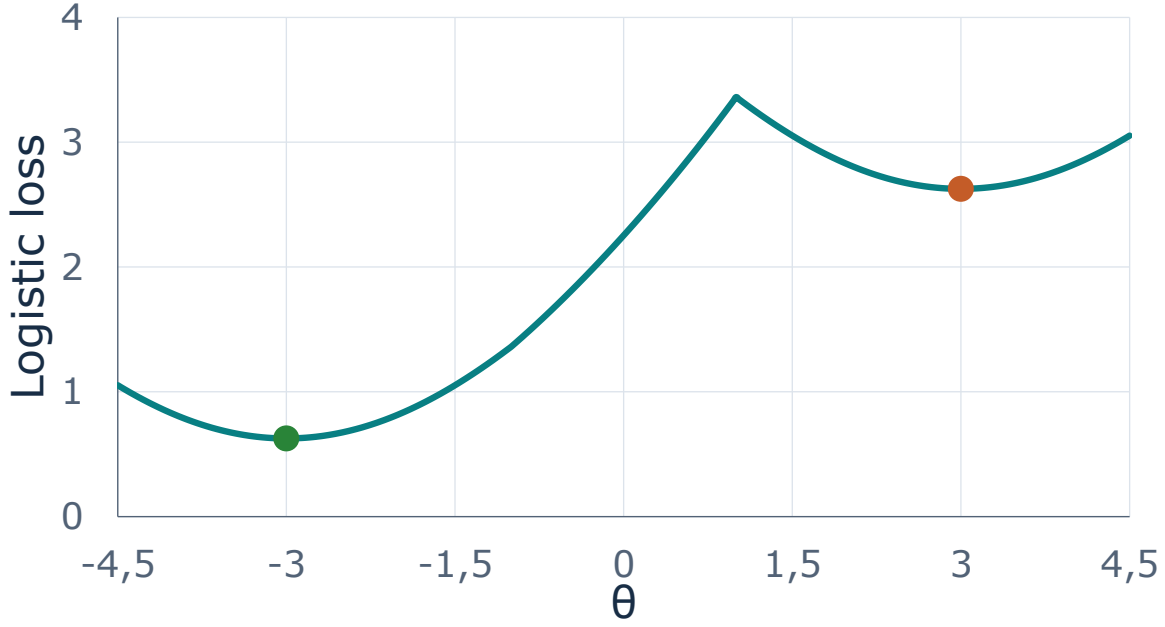
Local minimum at $\theta = +3$

No sufficiently small change improves the loss.

The same difficulty in classification

Two observations: $(x_1, y_1) = (-1, 0)$ and $(x_2, y_2) = (1, 1)$.

$$p_\theta(x) = \sigma\big(f_\theta(x)\big), \qquad \mathcal{L}_{\log}(\theta) = -\log\big(1 - p_\theta(-1)\big) - \log\big(p_\theta(1)\big)$$

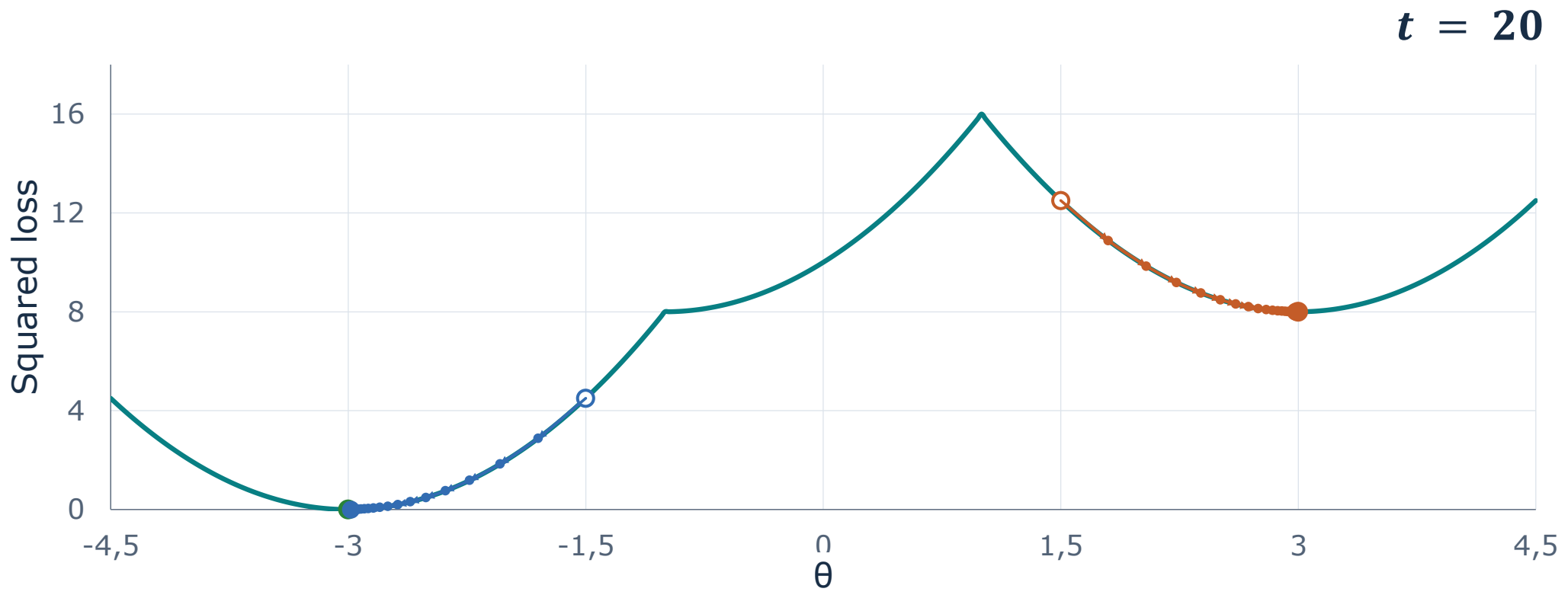


θ	Logistic loss	Mistakes
−3	0.627	0 of 2
+3	2.627	2 of 2

The local minimum can give much worse predictions than the global one.

Gradient descent: two starting points

Initial values $\theta^{(0)} = -1.5$ and $+1.5$. Learning rate $\eta = 0.05$.



Start at -1.5

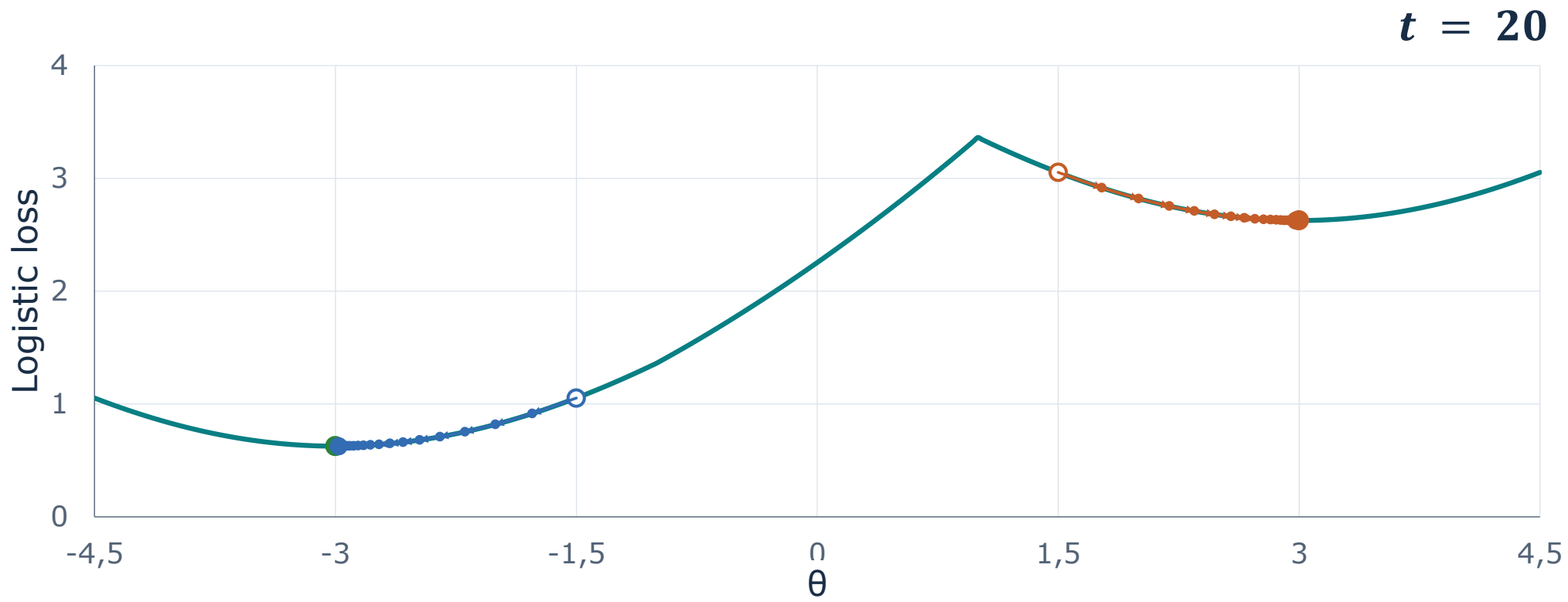
$\theta^{(20)} \approx -2.9827, \quad \mathcal{L}_{\text{sq}}\left(\theta^{(20)}\right) \approx 0.0006$

Start at $+1.5$

$\theta^{(20)} \approx 2.9827, \quad \mathcal{L}_{\text{sq}}\left(\theta^{(20)}\right) \approx 8.0006$

Gradient descent with logistic loss

Initial values $\theta^{(0)} = -1.5$ and $+1.5$. Learning rate $\eta = 0.5$.



Start at -1.5

$\theta^{(20)} \approx -2.9802, \quad \mathcal{L}_{\log}\left(\theta^{(20)}\right) \approx 0.6266$

Start at $+1.5$

$\theta^{(20)} \approx 2.9802, \quad \mathcal{L}_{\log}\left(\theta^{(20)}\right) \approx 2.6266$

Finding useful solutions in practice

These practical heuristics can help,
without guaranteeing convergence to a global minimum.

Try different initialisations

Different starting points can lead to different final parameters.
Keep track of which runs fit the training data better.

Change how the updates are computed

Stochastic Gradient Descent: use gradients from small random subsets.
This may help the iterates leave the region around a local minimum.

Consider increasing the width

A wider network can sometimes be easier to optimise.
Increasing width does not guarantee that gradient descent
reaches a global minimum.

Computing the gradient

Gradient descent requires the derivative of the loss
with respect to every weight and bias.

The network composes affine transformations and activation functions.
We then evaluate the loss.

Where these operations are differentiable,
we apply the chain rule through this composition.

ReLU is not differentiable at zero.
For parameter updates, we use 0 as its multiplier there.
This is a computational convention, not an ordinary derivative.

Backpropagation organises this calculation efficiently.

Next lecture: backpropagation.

Lecture notes and exercises

The notes for today's lecture are available on the course GitHub page.

They explain the material in more detail and include exercises.

I will post the exercise solutions before the next lecture.

github.com/semtm0051/2026_27

Scan for course materials